TRIBHUVAN UNIVERSITY

INSTITUTE OF ENGINEERING

THAPATHALI CAMPUS

A Project Report

On

Galaxy-Warrior-2D Space Shooter Game

Submitted By:

Binay Kumar Chand(L.E.082BCT02)

Submitted To:

BCT-II/I

Thapathali Campus

Kathmandu, Nepal

Under the Supervision of

Er. Kobid Karkee

March, 2026

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my instructor, Asst. Prof. Kobid Karkee, for providing guidance and support throughout the development of this project. His valuable suggestions and encouragement helped me understand the concepts of computer graphics and game development in depth.

I am also thankful to my institution, Thapathali Campus, for providing the resources and environment necessary for completing this project. Additionally, I would like to acknowledge the developers and contributors of Python and the Pygame library, whose tools made this project possible.

ABSTRACT

This project presents the design and development of a 2D space shooter game titled *Galaxy Warrior*, implemented using Python and the Pygame library. The primary goal of this project is to demonstrate fundamental concepts of computer graphics such as rendering, animation, real-time interaction, and collision detection in a practical and interactive way.

The game allows the player to control a spaceship that can move across the screen and shoot enemies. Various types of enemies are generated dynamically, each having different movement patterns and difficulty levels. Additionally, the game includes advanced features such as a boss fight system, health bar, scoring mechanism, and power-ups that enhance gameplay.

Overall, this project not only provides entertainment but also serves as a practical application of computer graphics concepts, helping to bridge the gap between theory and real-world implementation.

TABLE OF CONTENTS

1. Acknowledgement 1
2. Abstract 1
3. Introduction 2
4. Problem Statement 3
5. Objectives 3
6. Scope and Limitations 4
7. Tools and Technologies Used 4
8. Literature Review 5
9. Methodology / System Design 5
    1.1.3 System Design Overview
    1.1.4 Workflow Description
10. Implementation 6
    1.1.5 Main Components
    1.1.6 Key Features Implemented
11. Results / Output 7
    Game Interface
    Gameplay Output
    Boss Fight
12. Discussion 9
    1.1.7 Challenges Faced
    1.1.8 Analysis
13. Conclusion 10
14. Future Work 10
15. References 11
16. Appendix 12

INTRODUCTION

Computer Graphics is an important field that focuses on the creation, manipulation, and representation of visual images using computers. It is widely used in areas such as video games, simulations, animations, and virtual environments.

The *Galaxy Warrior* project is a 2D space shooter game designed to demonstrate how graphical systems work in real-time. The game uses the Pygame library, which provides tools for rendering graphics, handling user inputs, and managing animations efficiently.

In this project, the player interacts with a dynamic environment where objects continuously move, collide, and update. The game loop plays a crucial role in

ensuring that the system runs smoothly and responds instantly to user inputs. This project highlights how simple programming concepts can be combined to build a complex and interactive graphical application.

PROBLEM STATEMENT

The development of interactive graphical applications such as games involves multiple challenges, including real-time rendering, object management, and user interaction. Traditional theoretical knowledge of computer graphics is often insufficient to understand how dynamic systems operate in real-world applications.

This project addresses the challenge of designing and implementing a real-time 2D game system where multiple objects such as players, enemies, bullets, and power-ups interact simultaneously. The key problem is to ensure smooth animation, accurate collision detection, and responsive controls while maintaining performance efficiency.

Additionally, managing increasing game difficulty, implementing intelligent enemy behavior, and handling real-time updates through a continuous game loop adds complexity. This project aims to solve these problems by developing a structured and efficient game system using Python and Pygame.

OBJECTIVES

The main objectives of this project are explained below:

- To design and develop an interactive 2D game using Python and Pygame, focusing on real-time graphical rendering and smooth gameplay experience.
- To understand and implement core concepts of computer graphics such as object rendering, coordinate systems, and animation techniques.
- To apply object-oriented programming principles in designing game components like player, enemies, bullets, and power-ups.
- To implement collision detection mechanisms between different objects, ensuring realistic interactions and gameplay logic.
- To develop a scoring system, health management system, and level progression to make the game more engaging and challenging.
- To enhance programming skills by integrating logic, mathematics, and visualization into a single application.

SCOPE AND LIMITATIONS

1.1.1 Scope

- Development of a fully functional 2D space shooter game
- Implementation of real-time rendering and animation
- Use of keyboard-based user interaction
- Implementation of enemy AI and boss fight system
- Integration of scoring, health, and power-up systems
- Application of object-oriented programming in game design

1.1.2 Limitations

- Limited to 2D graphics (no 3D environment)
- No advanced physics engine
- Limited sound and visual effects
- No online multiplayer functionality
- Basic UI design without advanced menus

TOOLS AND TECHNOLOGIES USED

The development of this project required the use of several tools and technologies, each playing an important role in the overall implementation.

- PROGRAMMING LANGUAGE: PYTHON

Python is used because of its simplicity and readability, which makes it easier to implement complex game logic. It also supports rapid development and debugging.

- LIBRARY: PYGAME

Pygame is a powerful library used for creating 2D games. It provides functionalities such as:

  - Drawing shapes and objects on the screen
  - Handling keyboard inputs
  - Managing animations and frame updates

- DEVELOPMENT ENVIRONMENT: VISUAL STUDIO CODE

VS Code is used for writing and managing code efficiently. It provides features like syntax highlighting, debugging tools, and extensions.

- VERSION CONTROL: GIT AND GITHUB

Git is used to track changes in the project, while GitHub is used to store and share the project online.

LITERATURE REVIEW

Computer graphics plays a vital role in modern game development, simulations, and visual applications. According to standard literature, graphical systems rely on rendering techniques, coordinate systems, and animation to create dynamic visual environments.

Game development frameworks such as Pygame provide a simplified approach to building 2D games by handling low-level operations like rendering, input processing, and timing. Many modern games are built upon similar concepts, including game loops, collision detection, and event-driven programming.

The concept of real-time rendering ensures that objects are continuously updated and displayed on the screen. Collision detection algorithms are used to determine interactions between objects, which is essential in game logic.

This project is based on these core principles and demonstrates how theoretical concepts of computer graphics can be applied in a practical game development environment.

METHODOLOGY / SYSTEM DESIGN

The system follows a modular and layered architecture where each component operates independently but communicates with others through defined interfaces. The game loop acts as the core engine that continuously updates the system state.

The design ensures efficient memory management and smooth rendering. By separating input handling, processing, and rendering, the system achieves better performance and scalability. This modular approach also makes it easier to debug and extend the project.

The project follows a modular and structured design approach, where different components of the system are developed separately and then integrated together.

1.1.3 SYSTEM DESIGN OVERVIEW

The system is divided into the following major components:

- Game Loop – controls continuous execution of the game
- Entity System – manages player, enemies, bullets, and power-ups
- Input System – processes keyboard inputs
- Rendering System – displays graphics on the screen

- Collision System – detects interactions between objects

1.1.4 WORKFLOW DESCRIPTION

The working process of the system is explained below in both paragraph and step format:

The game begins by initializing the display window and loading necessary resources. After that, all game objects such as the player, enemies, and bullets are created. The system then enters the main game loop, which continuously runs until the game ends. During each iteration of the loop, user input is detected, object positions are updated, collisions are checked, and the updated graphics are rendered on the screen.

- Initialize the game window and load required resources
- Create game objects such as player, enemies, and bullets
- Enter the main game loop for continuous execution
- Detect user input for movement and shooting
- Update positions and states of all objects
- Check for collisions between objects
- Render updated graphics on the screen
- Repeat the process until the game ends

This approach ensures smooth performance and real-time updates.

IMPLEMENTATION

The implementation is carried out using Python and the Pygame library, which provides essential functionalities for game development. The project is structured into multiple modules, each responsible for a specific task.

The game utilizes object-oriented programming principles, where each game entity is defined as a class. This approach improves code organization and reusability.

The main game loop controls the execution flow by updating object states, processing user inputs, and rendering graphics at a constant frame rate. The use of frame rate control ensures smooth animation and consistent gameplay.

The implementation of the project is carried out in a modular way, where the code is divided into multiple files for better organization and readability.

1.1.5 MAIN COMPONENTS

The project consists of the following main modules:

- MAIN.PY

This file acts as the entry point of the program. It initializes the game and starts execution by calling the main game class.

- GAME.PY

This module contains the core logic of the game. It manages:

  - Game states such as menu, playing, and game over
  - Enemy spawning and level progression
  - Score calculation and display

- ENTITIES.PY

This file defines all game objects including:

  - Player class for movement and shooting
  - Enemy class with different behaviors
  - Boss class for advanced gameplay
  - Bullet class for shooting system

1.1.6 KEY FEATURES IMPLEMENTED

The project includes several important features, which are described below:

- Player movement using keyboard keys, allowing smooth navigation across the screen
- Shooting system where bullets are fired with controlled speed and direction
- Enemy AI with different movement patterns and difficulty levels
- Boss battle system that introduces a stronger enemy with complex attacks
- Health and lives system that determines game survival
- Power-ups that provide special abilities such as rapid fire and shield
- Explosion effects that enhance visual feedback
- Animated background using moving stars

RESULTS / OUTPUT

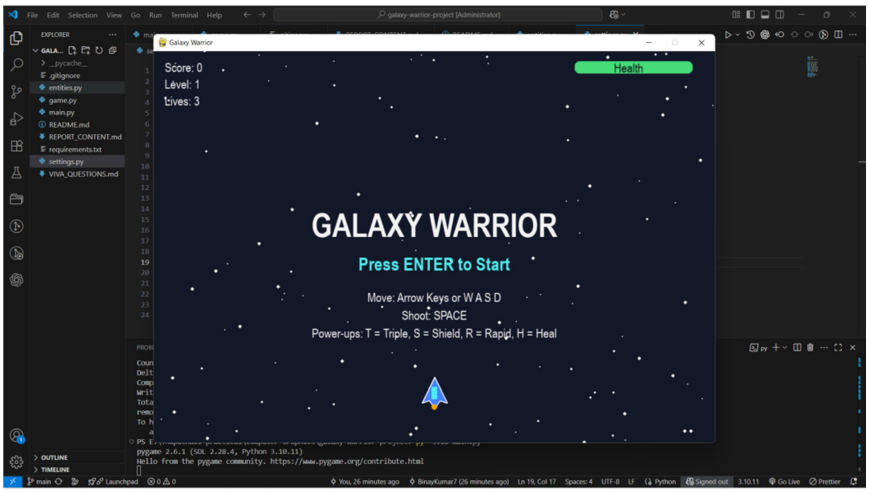
The final output of the project is a fully functional and interactive 2D game. The system successfully demonstrates real-time graphics rendering and user interaction.

The game displays:

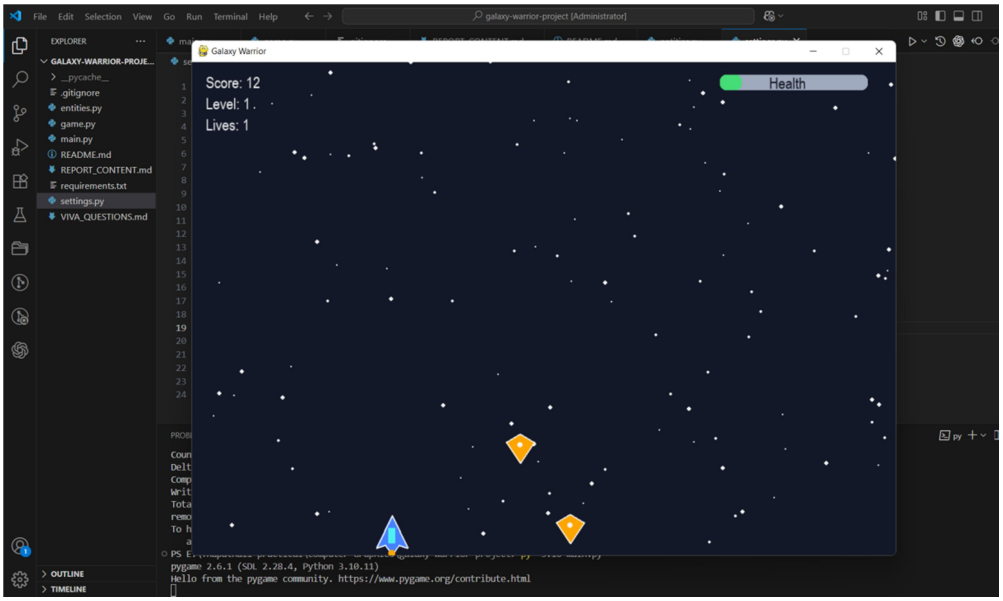
- A player-controlled spaceship at the bottom of the screen
- A dynamic star background that creates a space environment
- Enemies that spawn and move towards the player
- Bullets fired by both player and enemies
- A scoreboard showing score, level, and lives
- A health bar that updates in real time

The system responds accurately to user inputs and provides a smooth gaming experience without significant lag.

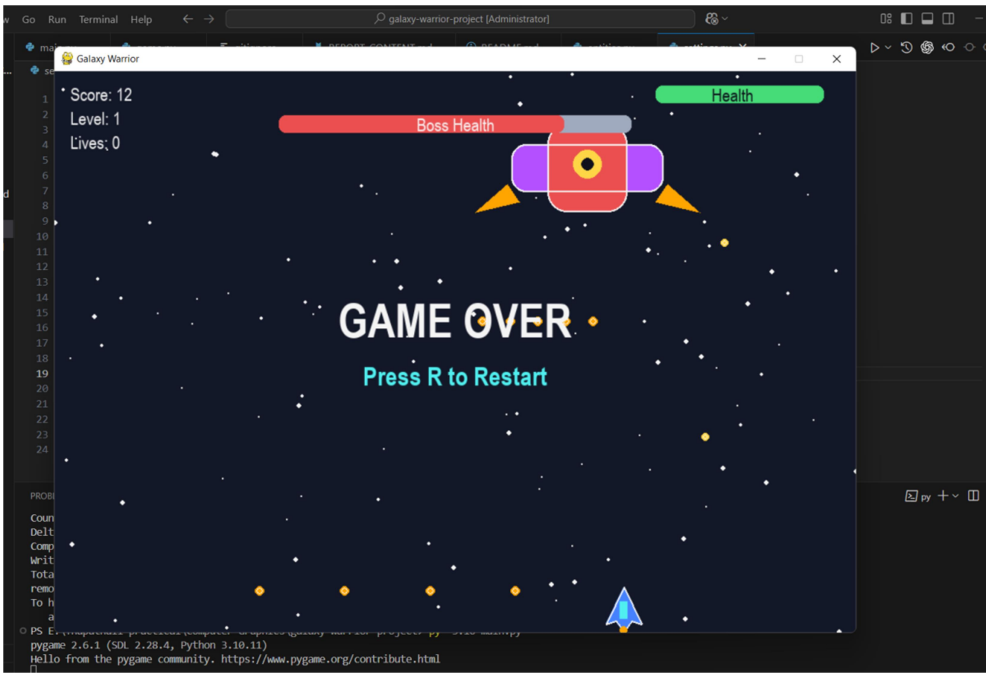
Game Interface



Gameplay Output



Boss Fight



DISCUSSION

The project highlights the importance of efficient game design and real-time processing. The use of Pygame simplifies graphical rendering but still requires careful handling of logic and performance.

One of the major strengths of the project is its modular structure, which allows easy modification and extension. However, limitations such as lack of advanced graphics and sound effects indicate areas for improvement.

The project also demonstrates how theoretical concepts such as coordinate systems, animation, and collision detection are applied in practical applications.

The project successfully demonstrates the practical application of computer graphics concepts. The use of Pygame simplifies the implementation of rendering and event handling.

1.1.7 CHALLENGES FACED

During the development of this project, several challenges were encountered:

- Managing multiple objects simultaneously in real time
- Ensuring smooth animation without performance issues

- Implementing accurate collision detection
- Debugging logical errors such as enemy spawning issues

1.1.8 ANALYSIS

Despite these challenges, the system performs efficiently and meets the project objectives. The modular design makes it easy to maintain and extend the project. The game provides a realistic and engaging experience, demonstrating the effectiveness of the implemented design.

CONCLUSION

In conclusion, the *Galaxy Warrior* project successfully demonstrates the design and development of a 2D space shooter game using Python and the Pygame library. The project effectively integrates fundamental concepts of computer graphics, including rendering, animation, collision detection, and real-time user interaction.

Through this project, a complete game system was developed where the player controls a spaceship, interacts with enemies, and progresses through different levels. The implementation includes various important features such as enemy spawning, shooting mechanisms, health and life management, power-ups, and a boss battle system. These features collectively contribute to an engaging and dynamic gameplay experience.

The project also highlights the importance of the game loop, which ensures continuous execution and real-time updates. Object-oriented programming played a significant role in organizing the code into modular components such as player, enemies, bullets, and game logic, making the system more structured and maintainable.

Additionally, this project provided practical experience in handling real-time events, managing multiple objects simultaneously, and debugging complex logic issues. Challenges such as enemy spawning errors and collision handling were successfully resolved, improving both the performance and reliability of the system.

Overall, the project not only fulfills its objectives but also enhances understanding of how graphical applications are built. It serves as a strong foundation for further exploration in game development, computer graphics, and interactive system design.

FUTURE WORK

The project can be further improved by adding additional features and enhancements:

- Adding sound effects and background music to improve user experience
- Using advanced graphics and images instead of simple shapes
- Implementing multiplayer mode for competitive gameplay
- Adding more levels and enemy types
- Improving user interface with menus and settings
- Converting the game into a standalone executable application

References

- Python Software Foundation, *Python Documentation*, Available at: https://www.python.org
→ Used for understanding Python syntax, functions, and programming concepts.
- Pygame Community, *Pygame Documentation*, Available at: https://www.pygame.org/docs
→ Used for implementing game development features such as rendering, input handling, and animations.
- Stack Overflow, *Programming Discussions and Solutions*, Available at: https://stackoverflow.com
→ Used for solving coding issues and debugging errors during development.
- GeeksforGeeks, *Python and Game Development Tutorials*, Available at: https://www.geeksforgeeks.org
→ Used for learning concepts related to game loops, collision detection, and logic building.
- YouTube Tutorials (Various Creators)
→ Used for visual understanding of game development techniques and Pygame implementation.

Appendix

main.py

```
from game import Game


if __name__ == '__main__':

    Game().run()
```

game.py

```
import random

import sys

import pygame

from settings import *

from entities import Player, Enemy, Boss, Bullet, Star, Explosion,
PowerUp


class Game:

    def __init__(self):
```

```
        pygame.init()

        pygame.display.set_caption(TITLE)

        self.screen = pygame.display.set_mode((WIDTH, HEIGHT))

        self.clock = pygame.time.Clock()

        self.font_small = pygame.font.SysFont('arial', 22)

        self.font_medium = pygame.font.SysFont('arial', 32, bold=True)

        self.font_large = pygame.font.SysFont('arial', 58, bold=True)

        self.reset()



    def reset(self):

        self.state = 'menu'

        self.player = Player()

        self.level = 1

        self.level_enemy_target = 20

        self.enemies_spawned = 0

        self.enemies_destroyed = 0

        self.spawn_delay = 400

        self.last_spawn = 0

        self.boss = None

        self.message = 'Press ENTER to Start'

        self.message_timer = 0
```

```
        self.stars = [Star() for _ in range(STAR_COUNT)]

        self.bullets = []

        self.enemy_bullets = []

        self.enemies = []

        self.explosions = []

        self.powerups = []



    def run(self):

        while True:

            dt = self.clock.tick(FPS)

            self.handle_events()

            self.update(dt)

            self.draw()



    def handle_events(self):

        for event in pygame.event.get():

            if event.type == pygame.QUIT:

                pygame.quit()

                sys.exit()

            if event.type == pygame.KEYDOWN:

                if self.state == 'menu' and event.key ==
pygame.K_RETURN:
```

```
                    self.start_game()

                elif self.state in ('game_over', 'victory') and
event.key == pygame.K_r:

                    self.reset()

                elif self.state == 'playing' and event.key ==
pygame.K_SPACE:

                    self.bullets.extend(self.player.shoot())



    def start_game(self):

        self.reset()

        self.state = 'playing'

        self.message = 'Level 1'

        self.message_timer = pygame.time.get_ticks()



    def spawn_enemy(self):

        if self.boss is not None:

            return

        if self.enemies_spawned < self.level_enemy_target:

            self.enemies.append(Enemy(self.level))

            self.enemies_spawned += 1



    def maybe_start_boss(self):
```

```
        if self.enemies_destroyed >= self.level_enemy_target and not
self.enemies and self.boss is None:

            self.boss = Boss(self.level)

            self.message = f'Boss Fight - Level {self.level}'

            self.message_timer = pygame.time.get_ticks()



    def next_level(self):

        self.level += 1

        if self.level > MAX_LEVEL:

            self.state = 'victory'

            return

        self.enemies_spawned = 0

        self.enemies_destroyed = 0

        self.level_enemy_target += 6

        self.spawn_delay = max(350, self.spawn_delay - 120)

        self.message = f'Level {self.level}'

        self.message_timer = pygame.time.get_ticks()



    def apply_powerup(self, kind):

        if kind == 'triple':

            self.player.power_timer = 8000

        elif kind == 'shield':
```

```
            self.player.shield_timer = 7000

        elif kind == 'rapid':

            self.player.rapid_timer = 7000

        elif kind == 'heal':

            self.player.health = min(self.player.max_health,
self.player.health + 30)



    def update(self, dt):

        for star in self.stars:

            star.update()



        for explosion in self.explosions[:]:

            explosion.update()

            if not explosion.alive:

                self.explosions.remove(explosion)



        if self.state != 'playing':

            return



        keys = pygame.key.get_pressed()

        self.player.move(keys)

        self.player.update_timers(dt)
```

```
        now = pygame.time.get_ticks()

        if now - self.last_spawn >= self.spawn_delay and
self.enemies_spawned < self.level_enemy_target:

            self.spawn_enemy()

            self.last_spawn = now



        self.maybe_start_boss()



        for enemy in self.enemies[:]:

            enemy.update()



            if enemy.can_shoot():

                self.enemy_bullets.append(enemy.shoot())



            if enemy.offscreen():

                self.enemies.remove(enemy)

                self.enemies_destroyed += 1



            elif enemy.rect.colliderect(self.player.rect):

                self.explosions.append(Explosion(enemy.x, enemy.y))

                self.enemies.remove(enemy)
```

```
            self.enemies_destroyed += 1

            dead = self.player.damage(25)

            if dead and self.player.lives < 0:

                self.state = 'game_over'



        if self.boss:

            self.boss.update()

            if self.boss.can_shoot():

                self.enemy_bullets.extend(self.boss.shoot())

            if self.boss.rect.colliderect(self.player.rect):

                dead = self.player.damage(50)

                if dead and self.player.lives < 0:

                    self.state = 'game_over'



        for bullet in self.bullets[:]:

            bullet.update()

            if bullet.offscreen():

                self.bullets.remove(bullet)

                continue



            hit_something = False
```

```
            for enemy in self.enemies[:]:

                if bullet.rect.colliderect(enemy.rect):

                    enemy.health -= 20

                    hit_something = True

                    self.explosions.append(Explosion(bullet.x,
bullet.y))



                    if enemy.health <= 0:

                        self.player.score += enemy.score

                        self.explosions.append(Explosion(enemy.x,
enemy.y))

                        if random.random() < 0.18:

                            self.powerups.append(PowerUp(enemy.x,
enemy.y))

                        self.enemies.remove(enemy)

                        self.enemies_destroyed += 1

                    break



            if not hit_something and self.boss and
bullet.rect.colliderect(self.boss.rect):

                hit_something = True

                self.explosions.append(Explosion(bullet.x, bullet.y))

                if self.boss.damage(18):

                    self.player.score += 250
```

```
                    self.explosions.append(Explosion(self.boss.x,
self.boss.y))

                    self.boss = None

                    self.next_level()



            if hit_something and bullet in self.bullets:

                self.bullets.remove(bullet)



        for bullet in self.enemy_bullets[:]:

            bullet.update()

            if bullet.offscreen():

                self.enemy_bullets.remove(bullet)

                continue



            if bullet.rect.colliderect(self.player.rect):

                self.enemy_bullets.remove(bullet)

                self.explosions.append(Explosion(bullet.x, bullet.y))

                dead = self.player.damage(12)

                if dead and self.player.lives < 0:

                    self.state = 'game_over'



        for powerup in self.powerups[:]:
```

```
            powerup.update()

            if powerup.offscreen():

                self.powerups.remove(powerup)

            elif powerup.rect.colliderect(self.player.rect):

                self.apply_powerup(powerup.kind)

                self.powerups.remove(powerup)



        if self.player.lives < 0:

            self.state = 'game_over'



    def draw_background(self):

        self.screen.fill(BLACK)

        pygame.draw.rect(self.screen, NAVY, (0, 0, WIDTH, HEIGHT))

        for star in self.stars:

            star.draw(self.screen)



    def draw_hud(self):

        score_text = self.font_small.render(f'Score:
{self.player.score}', True, WHITE)

        level_text = self.font_small.render(f'Level: {self.level}',
True, WHITE)

        lives_text = self.font_small.render(f'Lives: {max(0,
self.player.lives)}', True, WHITE)
```

```
        self.screen.blit(score_text, (20, 16))

        self.screen.blit(level_text, (20, 46))

        self.screen.blit(lives_text, (20, 76))



        pygame.draw.rect(self.screen, GRAY, (WIDTH - 250, 18, 210, 22),
border_radius=8)

        current_hp = max(0, self.player.health) /
self.player.max_health

        pygame.draw.rect(self.screen, GREEN, (WIDTH - 250, 18, int(210
* current_hp), 22), border_radius=8)



        hp_text = self.font_small.render('Health', True, BLACK)

        self.screen.blit(hp_text, (WIDTH - 180, 17))



        powers = []

        if self.player.power_timer > 0:

            powers.append('Triple Shot')

        if self.player.shield_timer > 0:

            powers.append('Shield')

        if self.player.rapid_timer > 0:

            powers.append('Rapid Fire')
```

```
        if powers:

            power_text = self.font_small.render(' | '.join(powers),
True, CYAN)

            self.screen.blit(power_text, (WIDTH // 2 -
power_text.get_width() // 2, 18))



        if self.boss:

            pygame.draw.rect(self.screen, GRAY, (WIDTH // 2 - 220, 55,
440, 22), border_radius=8)

            ratio = max(0, self.boss.health) / self.boss.max_health

            pygame.draw.rect(self.screen, RED, (WIDTH // 2 - 220, 55,
int(440 * ratio), 22), border_radius=8)

            text = self.font_small.render('Boss Health', True, WHITE)

            self.screen.blit(text, (WIDTH // 2 - text.get_width() // 2,
54))



    def draw_center_message(self, title, subtitle):

        title_surf = self.font_large.render(title, True, WHITE)

        sub_surf = self.font_medium.render(subtitle, True, CYAN)

        self.screen.blit(title_surf, title_surf.get_rect(center=(WIDTH
// 2, HEIGHT // 2 - 40)))

        self.screen.blit(sub_surf, sub_surf.get_rect(center=(WIDTH //
2, HEIGHT // 2 + 30)))



    def draw(self):

        self.draw_background()
```

```
        for powerup in self.powerups:
            powerup.draw(self.screen)
        for bullet in self.bullets:
            bullet.draw(self.screen)
        for bullet in self.enemy_bullets:
            bullet.draw(self.screen)
        for enemy in self.enemies:
            enemy.draw(self.screen)
        if self.boss:
            self.boss.draw(self.screen)
        for explosion in self.explosions:
            explosion.draw(self.screen)

        self.player.draw(self.screen)
        self.draw_hud()

        if self.state == 'menu':
            self.draw_center_message('GALAXY WARRIOR', 'Press ENTER to
Start')
            tips = [
                'Move: Arrow Keys or W A S D',
```

```
                'Shoot: SPACE',

                'Power-ups: T = Triple, S = Shield, R = Rapid, H =
Heal',

            ]

            for i, line in enumerate(tips):

                surf = self.font_small.render(line, True, WHITE)

                self.screen.blit(surf, surf.get_rect(center=(WIDTH //
2, HEIGHT // 2 + 90 + i * 32)))



        elif self.state == 'game_over':

            self.draw_center_message('GAME OVER', 'Press R to Restart')



        elif self.state == 'victory':

            self.draw_center_message('YOU WIN!', 'Press R to Play
Again')



        if self.message and pygame.time.get_ticks() -
self.message_timer < 2000 and self.state == 'playing':

            msg = self.font_medium.render(self.message, True, YELLOW)

            self.screen.blit(msg, msg.get_rect(center=(WIDTH // 2,
HEIGHT // 2)))



            pygame.display.flip()
```

entities.py

```
    import math
```

```python
import random

import pygame

from settings import *


class Star:

    def __init__(self):

        self.reset(initial=True)


    def reset(self, initial=False):

        self.x = random.randint(0, WIDTH)

        self.y = random.randint(0, HEIGHT) if initial else
random.randint(-40, -10)

        self.speed = random.uniform(1, 4)

        self.size = random.randint(1, 3)


    def update(self):

        self.y += self.speed

        if self.y > HEIGHT:

            self.reset()


    def draw(self, surface):

        pygame.draw.circle(surface, WHITE, (int(self.x), int(self.y)),
self.size)
```

```
class Bullet:

    def __init__(self, x, y, speed, owner='player', color=YELLOW,
radius=4, dx=0):

        self.x = x

        self.y = y

        self.speed = speed

        self.owner = owner

        self.color = color

        self.radius = radius

        self.dx = dx

        self.rect = pygame.Rect(self.x - radius, self.y - radius,
radius * 2, radius * 2)



    def update(self):

        self.y += self.speed

        self.x += self.dx

        self.rect.topleft = (self.x - self.radius, self.y -
self.radius)



    def draw(self, surface):

        pygame.draw.circle(surface, self.color, (int(self.x),
int(self.y)), self.radius)

        pygame.draw.circle(surface, WHITE, (int(self.x), int(self.y)),
max(1, self.radius - 2), 1)
```

```
    def offscreen(self):

        return self.y < -20 or self.y > HEIGHT + 20 or self.x < -20 or
self.x > WIDTH + 20



class Player:

    def __init__(self):

        self.width = 52

        self.height = 58

        self.x = WIDTH // 2

        self.y = HEIGHT - 90

        self.speed = PLAYER_SPEED

        self.health = 100

        self.max_health = 100

        self.score = 0

        self.lives = 3

        self.fire_delay = 260

        self.last_shot = 0

        self.power_timer = 0

        self.shield_timer = 0

        self.rapid_timer = 0

        self.rect = pygame.Rect(0, 0, self.width, self.height)
```

```
        self.update_rect()


    def update_rect(self):

        self.rect.center = (self.x, self.y)


    def move(self, keys):

        if keys[pygame.K_LEFT] or keys[pygame.K_a]:

            self.x -= self.speed

        if keys[pygame.K_RIGHT] or keys[pygame.K_d]:

            self.x += self.speed

        if keys[pygame.K_UP] or keys[pygame.K_w]:

            self.y -= self.speed

        if keys[pygame.K_DOWN] or keys[pygame.K_s]:

            self.y += self.speed


        self.x = max(self.width // 2, min(WIDTH - self.width // 2,
self.x))

        self.y = max(80, min(HEIGHT - self.height // 2, self.y))

        self.update_rect()


    def update_timers(self, dt):

        self.power_timer = max(0, self.power_timer - dt)
```

```
        self.shield_timer = max(0, self.shield_timer - dt)

        self.rapid_timer = max(0, self.rapid_timer - dt)



    def shoot(self):

        now = pygame.time.get_ticks()

        current_delay = 130 if self.rapid_timer > 0 else
self.fire_delay

        if now - self.last_shot < current_delay:

            return []

        self.last_shot = now



        bullets = [Bullet(self.x, self.y - 28, -BULLET_SPEED,
owner='player', color=YELLOW, radius=5)]

        if self.power_timer > 0:

            bullets.append(Bullet(self.x - 15, self.y - 20, -
BULLET_SPEED, owner='player', color=CYAN, radius=4, dx=-1.2))

            bullets.append(Bullet(self.x + 15, self.y - 20, -
BULLET_SPEED, owner='player', color=CYAN, radius=4, dx=1.2))

        return bullets



    def damage(self, amount):

        if self.shield_timer > 0:

            amount = max(0, amount // 3)

        self.health -= amount
```

```
        if self.health <= 0:
            self.lives -= 1
            self.health = self.max_health
            return True
        return False


    def draw(self, surface):
        points = [
            (self.x, self.y - 28),
            (self.x - 22, self.y + 18),
            (self.x - 10, self.y + 14),
            (self.x, self.y + 28),
            (self.x + 10, self.y + 14),
            (self.x + 22, self.y + 18),
        ]
        pygame.draw.polygon(surface, BLUE, points)
        pygame.draw.polygon(surface, WHITE, points, 2)
        pygame.draw.rect(surface, CYAN, (self.x - 5, self.y - 10, 10,
22))
        pygame.draw.circle(surface, ORANGE, (int(self.x), int(self.y +
26)), 5)
        if self.shield_timer > 0:
```

```
            pygame.draw.circle(surface, CYAN, (int(self.x),
int(self.y)), 42, 2)



class Enemy:

    def __init__(self, level):

        self.level = level

        self.kind = random.choice(['scout', 'fighter', 'tank'])

        self.x = random.randint(60, WIDTH - 60)

        self.y = random.randint(-200, -60)

        self.base_x = self.x

        self.wave_phase = random.uniform(0, math.pi * 2)

        self.rect = pygame.Rect(0, 0, 42, 42)



        if self.kind == 'scout':

            self.health = 20 + level * 4

            self.speed = 2.5 + level * 0.3

            self.score = 12

            self.color = RED

        elif self.kind == 'fighter':

            self.health = 35 + level * 6

            self.speed = 1.8 + level * 0.25

            self.score = 20
```

```
            self.color = PURPLE

        else:

            self.health = 55 + level * 10

            self.speed = 1.2 + level * 0.2

            self.score = 35

            self.color = ORANGE



        self.fire_delay = random.randint(400, 800)

        self.last_shot = pygame.time.get_ticks()

        self.update_rect()



    def update_rect(self):

        self.rect.center = (self.x, self.y)



    def update(self):

        self.y += self.speed

        self.x = self.base_x + math.sin(pygame.time.get_ticks() * 0.003
+ self.wave_phase) * 30

        self.update_rect()



    def can_shoot(self):

        now = pygame.time.get_ticks()
```

```
        if now - self.last_shot >= self.fire_delay and self.y > 0:

            self.last_shot = now

            return True

        return False



    def shoot(self):

        return Bullet(self.x, self.y + 18, ENEMY_BULLET_SPEED,
owner='enemy', color=RED, radius=4)



    def draw(self, surface):

        body = [(self.x, self.y + 18), (self.x - 20, self.y - 12),
(self.x, self.y - 24), (self.x + 20, self.y - 12)]

        pygame.draw.polygon(surface, self.color, body)

        pygame.draw.polygon(surface, WHITE, body, 2)

        pygame.draw.circle(surface, WHITE, (int(self.x), int(self.y -
8)), 4)



    def offscreen(self):

        return self.y > HEIGHT + 50



class Boss:

    def __init__(self, level):

        self.level = level

        self.x = WIDTH // 2
```

```
        self.y = 120

        self.width = 220

        self.height = 120

        self.rect = pygame.Rect(0, 0, self.width, self.height)

        self.max_health = 450 + level * 120

        self.health = self.max_health

        self.speed = 2.8

        self.direction = 1

        self.fire_delay = 800

        self.last_shot = 0

        self.entering = True

        self.arrive_y = 110

        self.angle = 0

        self.update_rect()


    def update_rect(self):

        self.rect.center = (self.x, self.y)


    def update(self):

        if self.entering:

            self.y += 1.5

            if self.y >= self.arrive_y:
```

```
                self.entering = False

        else:

            self.x += self.speed * self.direction

            if self.x < 140 or self.x > WIDTH - 140:

                self.direction *= -1

            self.angle += 0.04

        self.update_rect()



    def can_shoot(self):

        now = pygame.time.get_ticks()

        if now - self.last_shot >= self.fire_delay and not
self.entering:

            self.last_shot = now

            return True

        return False



    def shoot(self):

        bullets = []

        for dx in (-3.0, -1.5, 0, 1.5, 3.0):

            bullets.append(Bullet(self.x, self.y + 30,
ENEMY_BULLET_SPEED + 1, owner='enemy', color=ORANGE, radius=6, dx=dx))

        return bullets
```

```
    def draw(self, surface):

        x, y = self.x, self.y

        pygame.draw.rect(surface, PURPLE, (x - 95, y - 30, 190, 60),
border_radius=18)

        pygame.draw.rect(surface, RED, (x - 50, y - 55, 100, 110),
border_radius=24)

        pygame.draw.circle(surface, YELLOW, (int(x), int(y - 5)), 18)

        pygame.draw.polygon(surface, ORANGE, [(x - 100, y + 20), (x -
140, y + 55), (x - 85, y + 42)])

        pygame.draw.polygon(surface, ORANGE, [(x + 100, y + 20), (x +
140, y + 55), (x + 85, y + 42)])

        pygame.draw.rect(surface, WHITE, (x - 95, y - 30, 190, 60), 2,
border_radius=18)

        pygame.draw.rect(surface, WHITE, (x - 50, y - 55, 100, 110), 2,
border_radius=24)

        pygame.draw.circle(surface, BLACK, (int(x), int(y - 5)), 8)



    def damage(self, amount):

        self.health -= amount

        return self.health <= 0



class Explosion:

    def __init__(self, x, y):

        self.x = x

        self.y = y
```

```
        self.radius = 4

        self.max_radius = 30

        self.alive = True



    def update(self):

        self.radius += 2.8

        if self.radius >= self.max_radius:

            self.alive = False



    def draw(self, surface):

        pygame.draw.circle(surface, ORANGE, (int(self.x), int(self.y)),
int(self.radius), 3)

        pygame.draw.circle(surface, YELLOW, (int(self.x), int(self.y)),
max(1, int(self.radius * 0.55)), 2)



class PowerUp:

    TYPES = ['triple', 'shield', 'rapid', 'heal']



    def __init__(self, x, y):

        self.kind = random.choice(self.TYPES)

        self.x = x

        self.y = y

        self.speed = 2.5
```

```
        self.rect = pygame.Rect(0, 0, 24, 24)

        self.update_rect()



    def update_rect(self):

        self.rect.center = (self.x, self.y)



    def update(self):

        self.y += self.speed

        self.update_rect()



    def draw(self, surface):

        color_map = {

            'triple': CYAN,

            'shield': BLUE,

            'rapid': YELLOW,

            'heal': GREEN,

        }

        color = color_map[self.kind]

        pygame.draw.circle(surface, color, (int(self.x), int(self.y)),
12)

        pygame.draw.circle(surface, WHITE, (int(self.x), int(self.y)),
12, 2)

        label = self.kind[0].upper()
```

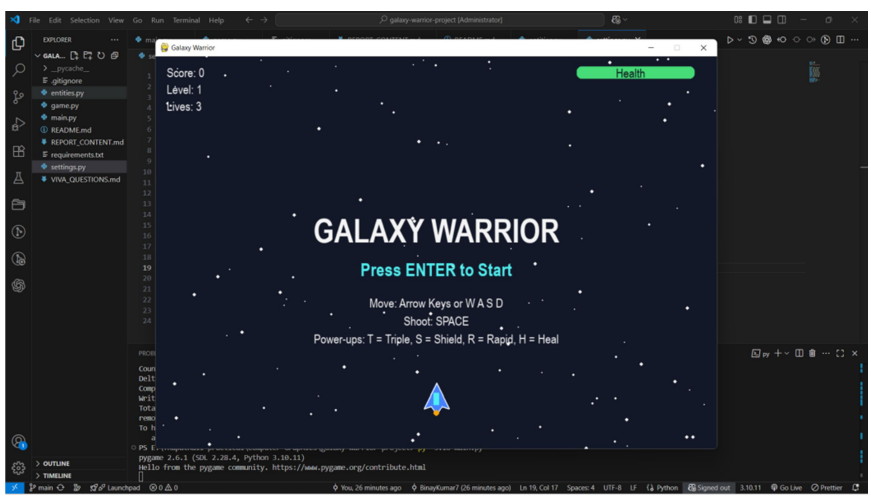
```
        font = pygame.font.SysFont('arial', 16, bold=True)

        text = font.render(label, True, BLACK)

        surface.blit(text, text.get_rect(center=(self.x, self.y)))



    def offscreen(self):

        return self.y > HEIGHT + 30
```

setting.py

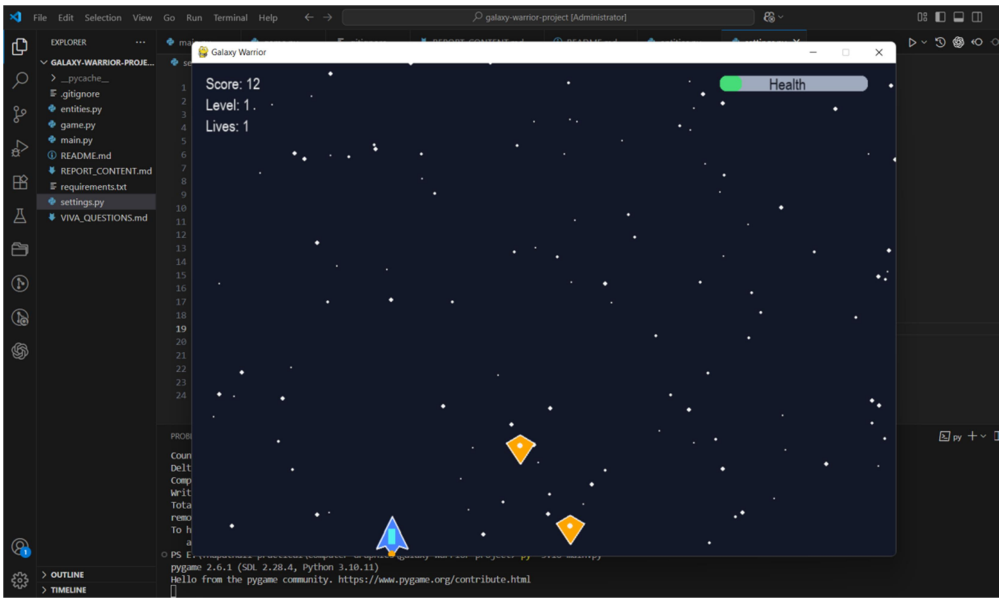
```
    WIDTH = 1000

HEIGHT = 700

FPS = 60

TITLE = 'Galaxy Warrior'



# Colors

WHITE = (245, 245, 245)

BLACK = (15, 18, 28)

NAVY = (19, 24, 40)

BLUE = (70, 130, 255)

RED = (235, 80, 80)

GREEN = (70, 220, 120)

YELLOW = (255, 215, 70)

ORANGE = (255, 165, 0)
```

```
PURPLE = (180, 80, 255)

CYAN = (80, 240, 240)

GRAY = (160, 170, 190)



PLAYER_SPEED = 6

BULLET_SPEED = 11

ENEMY_BULLET_SPEED = 6

STAR_COUNT = 120

MAX_LEVEL = 3
```

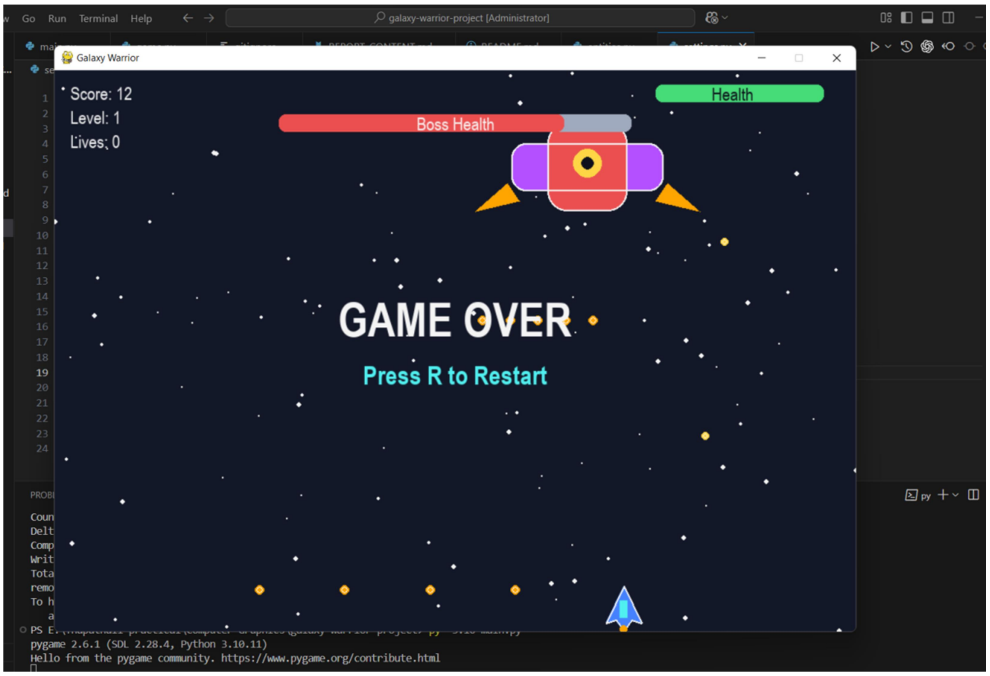
Game Interface



Gameplay Output


Galaxy Warrior
Score: 12
Level: 1
Lives: 1
Health
pygame 2.6.1 (SDL 2.28.4, Python 3.10.11)
Hello from the pygame community. https://www.pygame.org/contribute.html

Boss Fight


Galaxy Warrior
Score: 12
Level: 1
Lives: 0
Boss Health
Health
GAME OVER
Press R to Restart
pygame 2.6.1 (SDL 2.28.4, Python 3.10.11)
Hello from the pygame community. https://www.pygame.org/contribute.html

THANK YOU